Problem 3: Royal Archive Access Verification (Resolution Refutation)

Team GigaChad: Siddhi Singh (B24DS508), Anushka Gawande (B24CS007), Anu Yadav (B24EE007), Anjil Gupta (B24EE004), Bhumi Tapdiya (B24EE011)

Task 1: Knowledge Representation

Constants: mira, scroll_7 Variables: x, y

Predicates: Archivist(x), Verifies(x,y), Trusted(x), HasAccess(x,y), Sealed(y), Permitted(x,y), Inspects(x,y), FindsMark(x,y), Stored(y)

1. ∀x ∀y (Archivist(x) ∧ Verifies(x,y) → Trusted(x))
2. ∀x ∀y (Trusted(x) ∧ HasAccess(x,y) ∧ Sealed(y) → Permitted(x,y))
3. ∀x ∀y (Inspects(x,y) ∧ FindsMark(x,y) → Verifies(x,y))
4. ∀y (Stored(y) → Sealed(y))
5. Stored(scroll_7)
6. HasAccess(mira, scroll_7)
7. Inspects(mira, scroll_7) ∧ FindsMark(mira, scroll_7)
8. Archivist(mira)

Task 2: Negated Goal

¬Trusted(mira)

Task 3: Clause Form

No existential quantifiers are present, so no Skolemization is needed.

Clause	
C1	¬Archivist(x1) ∨ ¬Verifies(x1,y1) ∨ Trusted(x1)
C2	¬Trusted(x2) ∨ ¬HasAccess(x2,y2) ∨ ¬Sealed(y2) ∨ Permitted(x2,y2)
C3	¬Inspects(x3,y3) ∨ ¬FindsMark(x3,y3) ∨ Verifies(x3,y3)
C4	¬Stored(y4) ∨ Sealed(y4)
C5	Stored(scroll_7)
C6	HasAccess(mira, scroll_7)
C7.1	Inspects(mira, scroll_7)
C7.2	FindsMark(mira, scroll_7)
C8	Archivist(mira)
C9	¬Trusted(mira)

Task 5: Resolution Refutation

Step	Resolve	Substitution	Resolvent
R1	C9, C1	x1/mira	¬Archivist(mira) ∨ ¬Verifies(mira,y1)
R2	R1, C8	–	¬Verifies(mira,y1)
R3	R2, C3	x3/mira, y3/y1	¬Inspects(mira,y1) ∨ ¬FindsMark(mira,y1)
R4	R3, C7.1	y1/scroll_7	¬FindsMark(mira,scroll_7)
R5	R4, C7.2	–	[] (empty clause)

Task 6: Conclusion

The empty clause is derived, so the negated goal contradicts the knowledge base. Therefore Mira is trusted.

Screenshots

Steps 1 to 4

```
$ swipl royal_archive_access.pl


% Step 1: Predicate logic representation
  1: forall x forall y ((archivist(x) & verifies(x,y)) => trusted(x))
  2: forall x forall y ((trusted(x) & (has_access(x,y) & sealed(y))) => permitted(x,y))
  3: forall x forall y ((inspects(x,y) & finds_mark(x,y)) => verifies(x,y))
  4: forall y (stored_in_archive(y) => sealed(y))
  5: stored_in_archive(scroll_7)
  6: has_access(mira,scroll_7)
  7: (inspects(mira,scroll_7) & finds_mark(mira,scroll_7))
  8: archivist(mira)

% Step 2: Negation of goal
  9: ~trusted(mira)

% Step 3: Implication removal
  1: forall x forall y (~(archivist(x) & verifies(x,y)) | trusted(x))
  2: forall x forall y (~(trusted(x) & (has_access(x,y) & sealed(y))) | permitted(x,y))
  3: forall x forall y (~(inspects(x,y) & finds_mark(x,y)) | verifies(x,y))
  4: forall y (~stored_in_archive(y) | sealed(y))
  5: stored_in_archive(scroll_7)
  6: has_access(mira,scroll_7)
  7: (inspects(mira,scroll_7) & finds_mark(mira,scroll_7))
  8: archivist(mira)
  9: ~trusted(mira)

% Step 4: Negation movement
  1: forall x forall y ((~archivist(x) | ~verifies(x,y)) | trusted(x))
  2: forall x forall y ((~trusted(x) | (~has_access(x,y) | ~sealed(y))) | permitted(x,y))
  3: forall x forall y ((~inspects(x,y) | ~finds_mark(x,y)) | verifies(x,y))
  4: forall y (~stored_in_archive(y) | sealed(y))
  5: stored_in_archive(scroll_7)
  6: has_access(mira,scroll_7)
  7: (inspects(mira,scroll_7) & finds_mark(mira,scroll_7))
  8: archivist(mira)
  9: ~trusted(mira)
```

Steps 5 to 7

```
$ swipl royal_archive_access.pl   (continued)

% Step 5: Standardization
  1: forall x1 forall y1 ((~archivist(x1) | ~verifies(x1,y1)) | trusted(x1))
  2: forall x2 forall y2 ((~trusted(x2) | (~has_access(x2,y2) | ~sealed(y2))) | permitted(x2,y2))
  3: forall x3 forall y3 ((~inspects(x3,y3) | ~finds_mark(x3,y3)) | verifies(x3,y3))
  4: forall y4 (~stored_in_archive(y4) | sealed(y4))
  5: stored_in_archive(scroll_7)
  6: has_access(mira,scroll_7)
  7: (inspects(mira,scroll_7) & finds_mark(mira,scroll_7))
  8: archivist(mira)
  9: ~trusted(mira)

% Step 6: Skolemization (none needed)
  1: forall x1 forall y1 ((~archivist(x1) | ~verifies(x1,y1)) | trusted(x1))
  2: forall x2 forall y2 ((~trusted(x2) | (~has_access(x2,y2) | ~sealed(y2))) | permitted(x2,y2))
  3: forall x3 forall y3 ((~inspects(x3,y3) | ~finds_mark(x3,y3)) | verifies(x3,y3))
  4: forall y4 (~stored_in_archive(y4) | sealed(y4))
  5: stored_in_archive(scroll_7)
  6: has_access(mira,scroll_7)
  7: (inspects(mira,scroll_7) & finds_mark(mira,scroll_7))
  8: archivist(mira)
  9: ~trusted(mira)

% Step 7: CNF clauses
  C1: { ~archivist(x1) | ~verifies(x1,y1) | trusted(x1) }
  C2: { ~trusted(x2) | ~has_access(x2,y2) | ~sealed(y2) | permitted(x2,y2) }
  C3: { ~inspects(x3,y3) | ~finds_mark(x3,y3) | verifies(x3,y3) }
  C4: { ~stored_in_archive(y4) | sealed(y4) }
  C5: { stored_in_archive(scroll_7) }
  C6: { has_access(mira,scroll_7) }
  C7.1: { inspects(mira,scroll_7) }
  C7.2: { finds_mark(mira,scroll_7) }
  C8: { archivist(mira) }
  C9: { ~trusted(mira) }
```

Step 8: resolution proof (empty clause)

```
$ swipl royal_archive_access.pl   (continued)

% Step 8: Resolution refutation

Resolvent R1:
  C9: { ~trusted(mira) }
  C1: { ~archivist(x1) | ~verifies(x1,y1) | trusted(x1) }
  unifier: [x1/mira]
  => R1: { ~archivist(mira) | ~verifies(mira,y1) }

Resolvent R2:
  R1: { ~archivist(mira) | ~verifies(mira,y1) }
  C8: { archivist(mira) }
  unifier: []
  => R2: { ~verifies(mira,y1) }

Resolvent R3:
  R2: { ~verifies(mira,y1) }
  C3: { ~inspects(x3,y3) | ~finds_mark(x3,y3) | verifies(x3,y3) }
  unifier: [x3/mira,y3/y1]
  => R3: { ~inspects(mira,y1) | ~finds_mark(mira,y1) }

Resolvent R4:
  R3: { ~inspects(mira,y1) | ~finds_mark(mira,y1) }
  C7.1: { inspects(mira,scroll_7) }
  unifier: [y1/scroll_7]
  => R4: { ~finds_mark(mira,scroll_7) }

Resolvent R5:
  R4: { ~finds_mark(mira,scroll_7) }
  C7.2: { finds_mark(mira,scroll_7) }
  unifier: []
  => R5: []  (empty clause)

Empty clause derived. Contradiction found.
Therefore trusted(mira) is PROVED.
```